

Southeast University

Lab Report 05

HTTP, Fetch and Simple API Use

Course Title: Web and Internet Programming Lab

Course Code: CSE472



Submitted to

Mr Abid Ahmad

Faculty

Southeast University

Submitted by

Mst. Mahbuba Yasmin Maria

ID: 2022100010040

Department of Computer Science and Engineering

Southeast University

Date of Submission: 24 September 2026

1. Objectives

- Understand HTTP requests and inspect successful responses in the browser Network panel.
- Fetch a local JSON file asynchronously and display its properties on the webpage.
- Use a public API to retrieve a sample user's name and email address.
- Handle loading, HTTP error and network error states using JavaScript.

2. Tools used

HTML, CSS and JavaScript; Google Chrome and its Network panel; a local HTTP server; Git and GitHub. Node.js and Playwright were used for browser verification and additional screenshots. Python and ReportLab were used to prepare the PDF.

3. Work completed

The **lab-05-fetch-api** folder contains a workshop demonstration page with two buttons. **Load Workshop Details** calls **loadWorkshop()**, which requests **data/workshop.json**. A successful response supplies the workshop title, date, venue and number of seats to four matching HTML elements.

Load Sample User calls **loadSampleUser()** and requests the sample user endpoint at JSONPlaceholder. It displays the returned name and email address. Both functions show “Loading...” while waiting, check for HTTP status 200, parse the response using **response.json()** and update the page using **textContent**.

Non-200 responses produce a message containing the HTTP status code. The catch blocks display errors caused by a failed request or JSON parsing. CSS provides centered content, bordered cards and smaller padding on narrow screens.

The project source was inspected and tested for this report without changes. The current GitHub main branch was verified at commit **65445e8**, which includes the Lab 05 source files.

4. Implementation evidence — webpage

1100px x 1100px

SEU Tech Event 2026

Lab 05: HTTP, Fetch and Simple API Use

[Home](#) | [About](#) | [Schedule](#) | [Registration](#)

Workshop Fetch Demo

Use the buttons below to load local JSON and a public API.

Workshop Details

Click the button to load workshop information.

Load Workshop Details

Title: Web Development Workshop

Date: 15 October 2026

Venue: CSE Lab, Southeast University

Seats: 40

Workshop data loaded successfully.

Public API Practice

Load Sample User

Leanne Graham - Sincere@april.biz

SEU CSE — Web Programming Lab

Figure 1. Actual webpage after both buttons are clicked. The workshop details and public API user are displayed successfully.

The local JSON produces **Web Development Workshop, 15 October 2026, CSE Lab, Southeast University** and **40** seats. The public API returns **Leanne Graham - Sincere@april.biz**.

4. Implementation evidence — HTTP 200

The Chrome Network panel was opened before running the requests. The Fetch/XHR filter isolates the two requests initiated by the JavaScript functions.

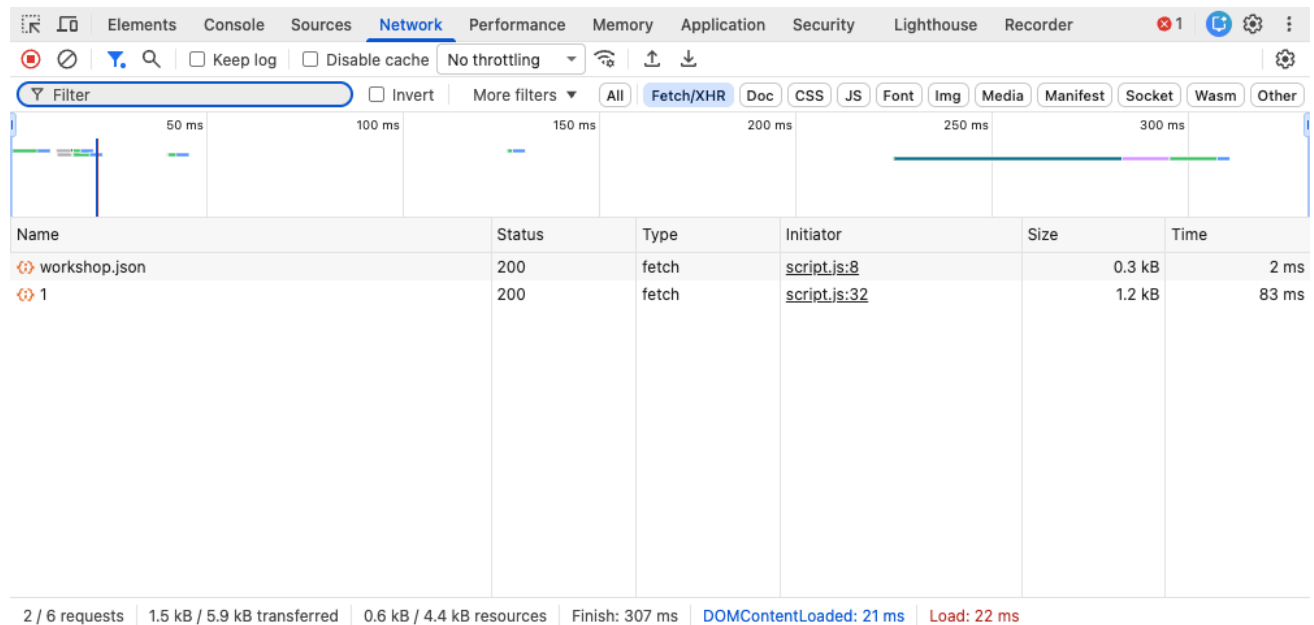


Figure 2. Actual Chrome Network panel capture. Both workshop.json and the API request named 1 have Status 200 and Type fetch.

Request	Method	Observed status
data/workshop.json	GET	200
https://jsonplaceholder.typicode.com/users/1	GET	200

Interpreting the result

A status of **200** confirms that the server successfully handled the request. The request named **1** is the last path segment of the public API URL. After each successful response, the script parses the JSON body and displays the selected properties.

The local file was served through **http://127.0.0.1:8765/** during verification. Serving the project over HTTP allows the relative path **data/workshop.json** to resolve against the webpage's origin. The public API requires internet access.

5. Representative code — JavaScript

```
// Lab 05: HTTP, Fetch and Simple API Use

async function loadWorkshop() {
  const msg = document.getElementById("loadMessage");
  msg.textContent = "Loading...";

  try {
    const response = await fetch("data/workshop.json");

    if (response.status === 200) {
      const workshop = await response.json();

      document.getElementById("workshopTitle").textContent = workshop.title;
      document.getElementById("workshopDate").textContent = workshop.date;
      document.getElementById("workshopVenue").textContent = workshop.venue;
      document.getElementById("workshopSeats").textContent = workshop.seats;

      msg.textContent = "Workshop data loaded successfully.";
    } else {
      msg.textContent = "Could not load workshop data. Status: " + response.status;
    }
  } catch (err) {
    msg.textContent = "Error: " + err.message;
  }
}

async function loadSampleUser() {
  const output = document.getElementById("apiUser");
  output.textContent = "Loading...";

  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
    if (response.status === 200) {
      const user = await response.json();
      output.textContent = user.name + " - " + user.email;
    } else {
      output.textContent = "Could not load API data. Status: " + response.status;
    }
  } catch (err) {
    output.textContent = "Error: " + err.message;
  }
}
```

Figure 3. Supplied screenshot of `js/script.js`, checked against the source file. Both asynchronous `fetch` functions, status checks, DOM updates and catch blocks are shown.

loadWorkshop() writes four JSON properties into the page and confirms successful loading. **loadSampleUser()** combines the user's name and email. Each function handles non-200 responses separately from exceptions.

5. Representative code — HTML and JSON

```
<section id="home">
  <h2>Workshop Fetch Demo</h2>
  <p>Use the buttons below to load local JSON and a public API.</p>

  <section class="card">
    <h3>Workshop Details</h3>
    <p>Click the button to load workshop information.</p>

    <button type="button" onclick="loadWorkshop()">Load Workshop Details</button>

    <p><strong>Title:</strong> <span id="workshopTitle">Not loaded</span></p>
    <p><strong>Date:</strong> <span id="workshopDate">Not loaded</span></p>
    <p><strong>Venue:</strong> <span id="workshopVenue">Not loaded</span></p>
    <p><strong>Seats:</strong> <span id="workshopSeats">Not loaded</span></p>
    <p id="loadMessage"></p>
  </section>

  <section class="card">
    <h3>Public API Practice</h3>
    <button type="button" onclick="loadSampleUser()">Load Sample User</button>
    <p id="apiUser">No API data loaded yet.</p>
  </section>
</section>
```

Figure 4. Browser-rendered screenshot of the actual `index.html` excerpt: click handlers, result elements and feedback containers.

```
{
  "title": "Web Development Workshop",
  "date": "15 October 2026",
  "venue": "CSE Lab, Southeast University",
  "seats": 40
}
```

Figure 5. Browser-rendered screenshot of the actual `data/workshop.json` file.

The span IDs match the IDs used by `getElementById()`. JSON property names and string values use double quotes; `seats` is stored as a number. The JSON file contains data, while the JavaScript controls when it is fetched and displayed.

6. Key concept explanation

Concept	Use in this project
HTTP GET	<code>fetch(url)</code> sends a GET request when no method option is supplied. The demo reads data rather than submitting a registration.
HTTP status	<code>response.status === 200</code> gates JSON parsing and successful output. Other HTTP statuses are reported to the user.
Fetch API	<code>fetch()</code> returns a promise for a <code>Response</code> . HTTP errors such as 404 still return a response, so the code checks its status explicitly.
async / await	The async functions use <code>await</code> to wait for a response and then its parsed JSON body. The page remains usable while the request is pending.
JSON parsing	<code>response.json()</code> converts JSON text into a JavaScript value. The functions read workshop and user properties from that value.
DOM updates	<code>getElementById()</code> locates the target element. <code>textContent</code> inserts data as text and replaces the previous content.
Error handling	<code>try/catch</code> handles rejected requests and JSON parsing failures. The <code>else</code> branches handle non-200 HTTP responses.
Local file and public API	The workshop file is served by the same local server as the page. The user data comes from a separate HTTPS service.
Loading feedback	Each function displays <code>Loading...</code> before awaiting the request, then replaces it with results or an error message.

The demo performs two independent read operations. It does not reserve seats, save data to `localStorage`, or send form data to a server.

7. Testing and observations

Automated browser checks were run against the unchanged project. Success cases used the actual local JSON file and public API. Error cases used browser request interception and are identified below as simulations.

Test	Observed result	Status
Initial page	Workshop fields show Not loaded; API output shows its initial message.	Pass
Load local JSON	All expected workshop values appear; request returns HTTP 200.	Pass
Load public API	Leanne Graham - Sincere@april.biz appears; request returns HTTP 200.	Pass
Refresh the page	Fetches workshop values reset to the initial state.	Pass
Simulated local HTTP 404	Could not load workshop data. Status: 404	Pass
Simulated API HTTP 503	Could not load API data. Status: 503	Pass
Simulated local request failure	Workshop feedback begins with Error:.	Pass
Simulated API request failure	API feedback begins with Error:.	Pass
Simulated malformed local JSON	The catch block displays Error: feedback.	Pass
390px viewport	No horizontal page overflow is detected.	Pass
JavaScript runtime	No uncaught JavaScript errors occur during the checks.	Pass

Result: 11 of 11 checks passed. No source-code corrections were made. These checks cover the demonstration's main success and failure paths; they are not a cross-browser compatibility test.

8. Reflection and conclusion

This exercise demonstrates the difference between receiving an HTTP response and successfully using its data. A request can complete with a non-200 status, while a 200 response can still contain invalid JSON. Explicit status checks and try/catch therefore serve different purposes.

Separating workshop data from the HTML makes it possible to update the displayed details without rewriting the page. Loading feedback also helps explain the delay between a button click and the arrival of data. Inspecting the Network panel connects the visible result to the underlying HTTP request.

Limitations and possible improvements

The demo accepts only status 200, assumes the expected JSON properties exist and depends on the public API being available. Previously loaded workshop values remain visible if a later request fails. Possible improvements include validating the response structure, clearing stale values on failure and disabling buttons while requests are pending. The About, Schedule and Registration navigation links have no matching sections in this Lab 05 page.

Source files and GitHub folder

Files: `index.html`, `css/style.css`, `js/script.js` and `data/workshop.json` in `lab-05-fetch-api/`.

<https://github.com/Mariya10040/Web-Internet-Programming-Lab/tree/main/lab-05-fetch-api>

The remote main branch matched the local Lab 05 commit at report preparation. Submission references: Screenshot_20260924-223557.jpg and CSE472_Lab04_2022100010040.pdf. The latter supplied the cover details and report format.

AI-use declaration

AI assistance (Codex) was used to inspect the existing project, run browser checks, capture additional evidence, draft the explanations and reflection, and assemble this PDF. The Lab 05 application source was not changed during report preparation. The report's test results come from executed checks; the HTTP and network error cases were simulated. Prior AI involvement in creating the original application was not established.

Submission checklist

One correctly named PDF; direct GitHub folder link; source code confirmed on GitHub; representative code screenshots; working webpage screenshot; Network panel showing HTTP 200; testing and reflection; AI-use declaration. The submission date is 24 September 2026.